

Faculty of Science and Technology,
UNIVERSITY OF MACAU

ECEN3024 FUNDAMENTALS OF INTERNET OF THINGS

FALL 2025/2026



Project Report

GestureFace Smart Control:
A Vision-Based Home Automation System

Name & Student no.: Wu Yu Fei (D-C2-2886-2) & Fong Pok Wai (D-C3-2508-6)

Faculty: Department of Electrical and Computer Engineering

Email: dc22886@um.edu.mo, dc32508@um.edu.mo

• Report Outline

I.	Introduction	Written by -- Wu Yufei (DC228862)
II.	Discussion of the Project	
	A. Task allocation	
	B. Raspberry Pi Part	Wu Yufei (DC228862)
	C. Android Studio Part	Fong Pokwai (DC325086)
	D. 3D Printed Case	Fong Pokwai (DC325086)
III.	Conclusions	Wu Yufei (DC228862)
IV.	Video Footage	

I. INTRODUCTION

People's demand of greater convenience and higher security level for the smart home control is growing steadily nowadays. On the one hand, the customers pursue the experience of direct control with non-physical contact. On the other hand, permission management for device usage and children's personal safety are also receiving increasing attention. The smart control of hazardous furniture like the cooking stove through identity recognition can effectively prevent children's misuse. In this case, the smart home control system supporting multi-recognition, also with differentiated permission control becomes the mainstream need for the modern family.

Our project of a vision-based home automation system that supports real-time monitoring, also the dual-recognition through face and gesture to control different objects is designed to meet this demand. Through the usage of Raspberry Pi as the central controller that managing all hardware components and the platform of Android Studio, this solution bridges affordable hardware with modern software development. It can transfer the low-latency video stream from the USB camera to the phone app, achieving various devices control and security alarm through the combination of face recognition and gesture control technologies. Thus, provide a convenient, intuitive, and secure smart home experience for the users.

II. DISCUSSION OF THE PROJECT

GestureFace Smart Control: A Vision-Based Home Automation System

A. Task allocation

The conception and development of this project were largely accomplished by us together, where both of us fully involved in every aspect of the project. During the debugging phase, Wu Yufei (DC228862) is primarily responsible for hardware adjustments related to the Raspberry Pi and Feng Pokwai (DC325086) is responsible for app Ui design on the android studio and video stream transmission, also with his creative design of the 3D printed case for the project.

B. Raspberry Pi Part

For the hardware part, we integrated 7 components to achieve multifunction: the USB Camera for live video monitoring, the DC motor for the fan control simulation, the touch sensor to simulate door bell reaction with the user, the LCD display for system status alerts, the LED for visual feedback, the servo motor serves as the door lock, finally the buzzer used for the doorbell and security alarm.

The outfit of our system is shown in Figure 1. We set the user “jaryd” as the master, who has the highest control level for all the devices (Five fingers up to control the door lock, 1 or 2 fingers up to control the on/off function of the fan...). Any user is allowed to control the LED light with their gesture (2/3/4 fingers up for blue, green, red light accordingly). In addition, at the outside of the door, people can activate the doorbell through the touch sensor. However, when people are recognized as “Unknown” and trigger the command to control any device, then the alarm will ring and the system will enter the alarm mode



Fig.1. The outfit of the project.

With those components, we designed mainly 5 functions for our project by combining various algorithms and models invoked, which are shown in Figure 1.

```
Core Computer Vision & AI Libraries

import cv2          # Real-time video processing, camera control, image rendering
import mediapipe as mp # Hand gesture recognition (21 landmark points detection)
import face_recognition # Face detection and identification using deep learning
import numpy as np   # Mathematical operations & array processing for image data
import pickle        # Loading pre-trained face encodings from saved files

Hardware Control & GPIO Libraries

from gpiozero import LED, Servo, Buzzer, AngularServo # Simplified Raspberry Pi hardware control
import RPi.GPIO as GPIO # Low-level GPIO pin management
import smbus        # I2C communication for LCD display

System & Networking Libraries

import threading      # Concurrent execution (non-blocking operations)
from flask import Flask, Response # Web server for live video streaming
import time, math, signal, sys # System utilities & timing control
```

Fig.2. The imported libraries.

To begin with, the function of Identify Verification is achieved by face recognition technology. To identify a specific person, it is needed to first upload a few photos to the system in advance. The face in the photos will be detected through the “HOG” model, which is fast enough to process, then extracted as a facial encoding with 128-dimensional feature vector by a training model. In this case, when the main code is processing, this facial encoding will be preloaded and calculate the matching parameters using Euclidean distance. If the parameters are high enough (in our setting, >0.65), then the recognition of the face is realized. The detailed code is shown below in Figure 2.

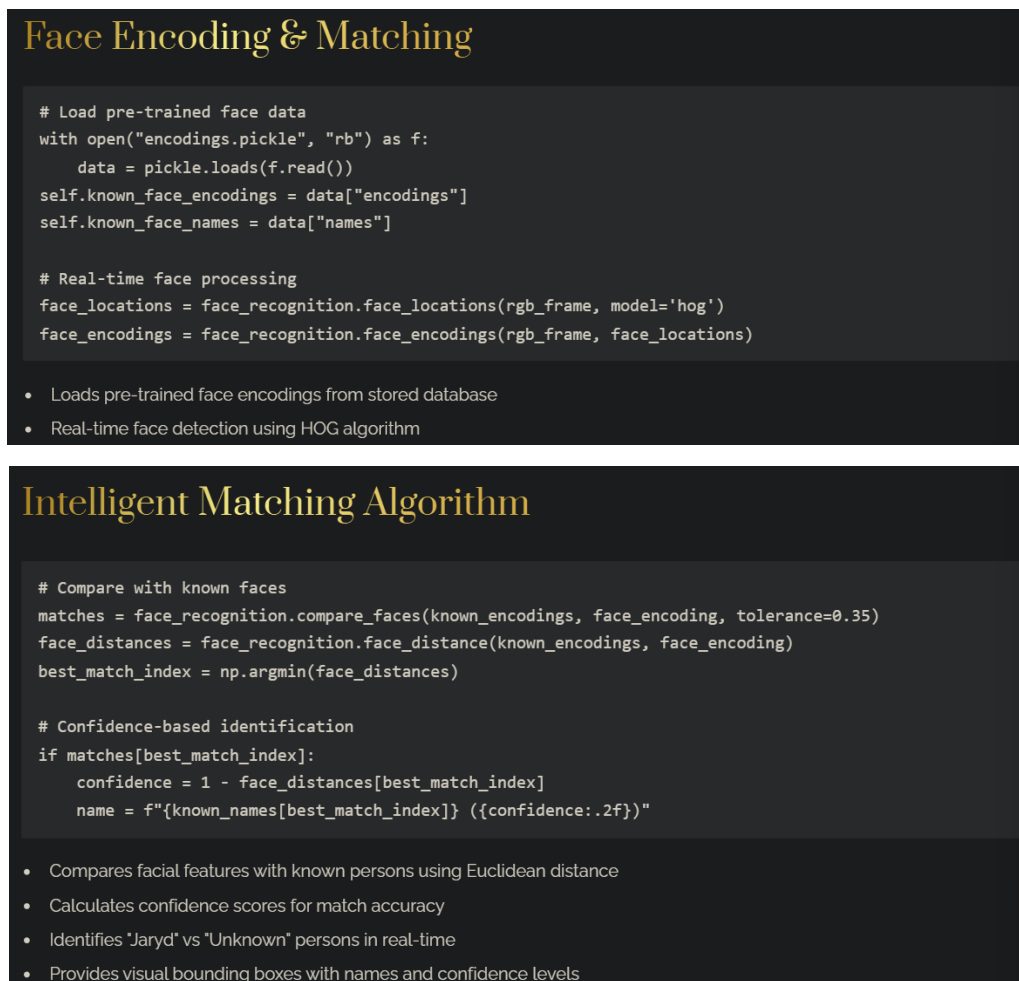
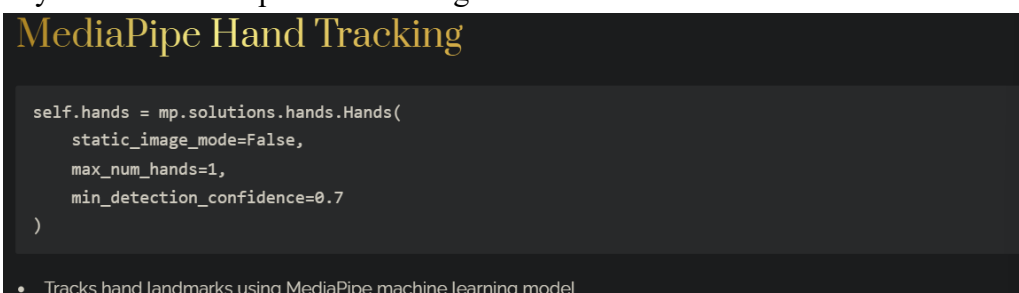


Fig.3. Identify Verification achieved by face recognition.

Moreover, our project realized intent confirmation through gesture recognition, where the machine learning model of Mediapipe is imported to achieve hand tracking. Since we use the method to control device by counting how many fingers are up, we also improved the finger counting algorithm with multi-angle adaptability to realize more precise counting.



Advanced Finger Counting Algorithm

```
def count_fingers(self, hand_landmarks, image_shape):
    thumb_score = 0
    if angle > 160: thumb_score += 1
    if dist_tip_wrist > dist_ip_wrist * 1.05: thumb_score += 1
    return thumb_score >= 2 # 2/3 conditions must be met
```

- Counts extended fingers using multi-criteria algorithm:
 - Angle-based: Joint angles > 160 degrees
 - Distance-based: Finger segment length ratios (tip-to-pip vs pip-to-mcp ratios)
 - Position-based: Y-coordinate comparisons

Fig.4. Intent Confirmation achieved by gesture recognition.

The Function of Security Protection and Device Control are easily implemented by reasonably allocate the gpio channel and the coding of multithreaded execution, whose content are clearly shown in Figure 5 and 6.

Unknown Person Detection

```
def is_unknown_present(self):
    return any("unknown" in name.lower() for
name in self.face_names)

def update_alarm(self, unknown_detected,
fingers_detected):
    if unknown_detected and fingers_detected:
        self.activate_alarm()
```

Smart Alarm Activation

```
def activate_alarm(self):
    if not self.is_active:
        print("ALERT: Unknown person detected
with finger gesture!")
        self.buzzer.on()
        self.is_active = True
```

- Detects unknown persons in camera view
- Requires dual confirmation: Unknown face + hand gestures present
- Activates buzzer alarm only when both conditions met
- Prevents false alarms from passive bystanders
- Automatic alarm deactivation when threat conditions clear
- LCD alert displays for immediate visual warnings

Fig.5. Security Protection with threat detection.

Doorbell with Touch Sensor

```
class Doorbell:
    def __init__(self, touch_pin=18):
        GPIO.setup(touch_pin, GPIO.IN, pull_up_down=GPIO.PUD_UP)

    def check_doorbell(self):
        # Edge detection: HIGH to LOW transition
        if (self.last_sensor_state == GPIO.HIGH and
            current_sensor_state == GPIO.LOW):
            return True # Touch detected!
```

Servo-based Door Lock System

```
class DoorController:
    def _servo_operation_thread(self, operation):
        if operation == "unlock":
            self.servo.angle = 90 # 90 degrees = Unlocked
            self.is_locked = False
        elif operation == "lock":
            self.servo.angle = 0 # 0 degrees = Locked
            self.is_locked = True
```

Gesture-controlled Fan

```
# Fan control logic
if jaryd_detected and finger_count == 1 and not fan_motor.is_on:
    fan_motor.turn_on()    # 1 finger = Fan ON
elif jaryd_detected and finger_count == 2 and fan_motor.is_on:
    fan_motor.turn_off()   # 2 fingers = Fan OFF
```

Doorbell System:

- Touch sensor detection on GPIO pin 18
- Audible doorbell ring with buzzer
- Visitor alert display on LCD screen

Door Lock Control:

- Servo motor operation for physical locking mechanism
- Automatic unlocking: Jaryd + 5 fingers gesture
- Timer-based relocking after 5 seconds for security
- Non-blocking thread for smooth operation

Fan Control:

- Gesture-based operation: 1 finger - ON, 2 fingers - OFF
- Identity-restricted: Only works when Jaryd detected
- Cooldown protection prevents rapid cycling

Fig.6. Device Control with smart automation.

Finally is the function of remote monitoring. On the one hand, we improve the FPS by video size and scanning frequency. Since we find the FPS will drop incredibly low when the facial recognition and hand detection ran simultaneously, we creatively enabling odd frames to run facial recognition while even frames to run hand detection, which proved to be a very effective method. On the other hand, we create a channel for pi and device app to transfer the video, which will be clearly explained in the android studio part below.

Flask Web Server Implementation

```
class VideoStreamServer:
    def __init__(self, host='0.0.0.0', port=5000):
        self.app = Flask(__name__)

        @self.app.route('/video_feed')
        def video_feed():
            return Response(self.generate_frames(),
                            mimetype='multipart/x-mixed-replace; boundary=frame')
```

MJPEG Stream Optimization

```
def generate_frames(self):
    while self.is_running:
        ret, buffer = cv2.imencode('.jpg', self.frame, [
            cv2.IMWRITE_JPEG_QUALITY, 60,      # Balanced quality
            cv2.IMWRITE_JPEG_OPTIMIZE, 1       # Compression optimization
        ])
        yield (b'--frame\r\n' + frame_bytes + b'\r\n')
```

- Flask web server hosts live video stream
- MJPEG streaming optimized for bandwidth efficiency
- Remote access from any device on network
- Real-time status overlay shows all system information
- Non-blocking frame updates maintain system performance
- Web interface displays FPS, face recognition, gestures, and security status

Fig.7. Remote Monitoring achieved by live streaming.

C. Android Studio Part

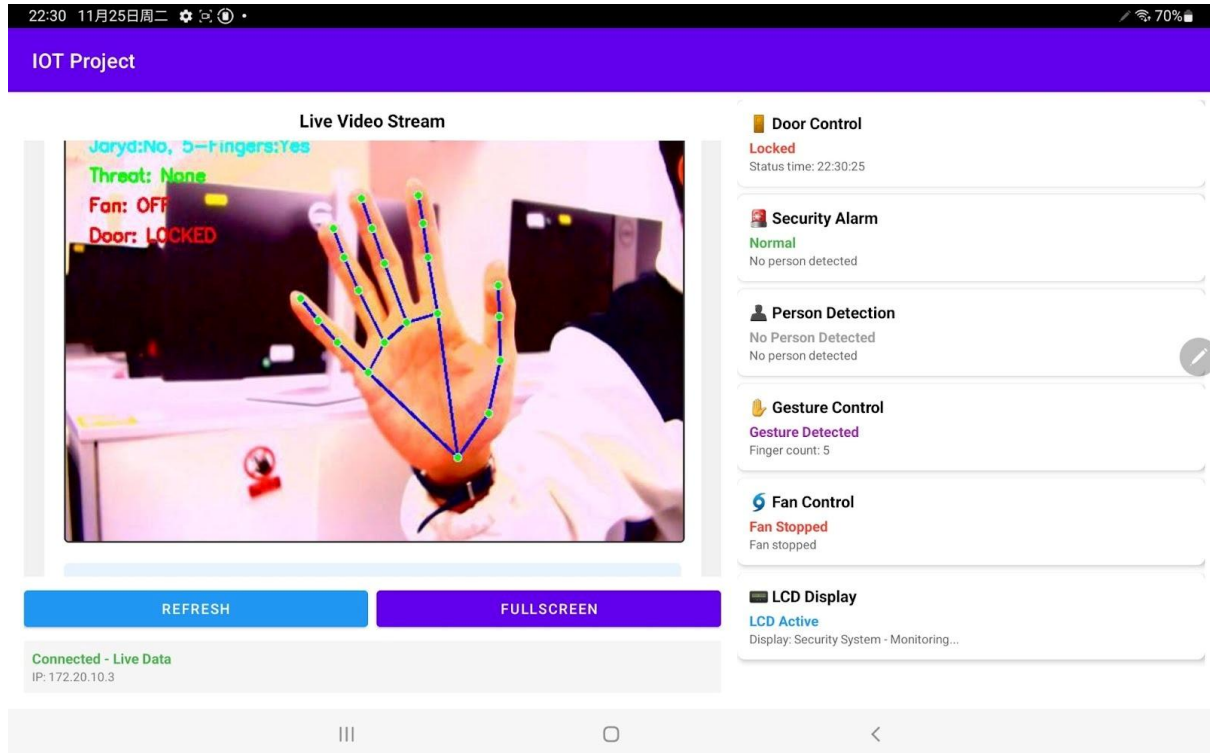
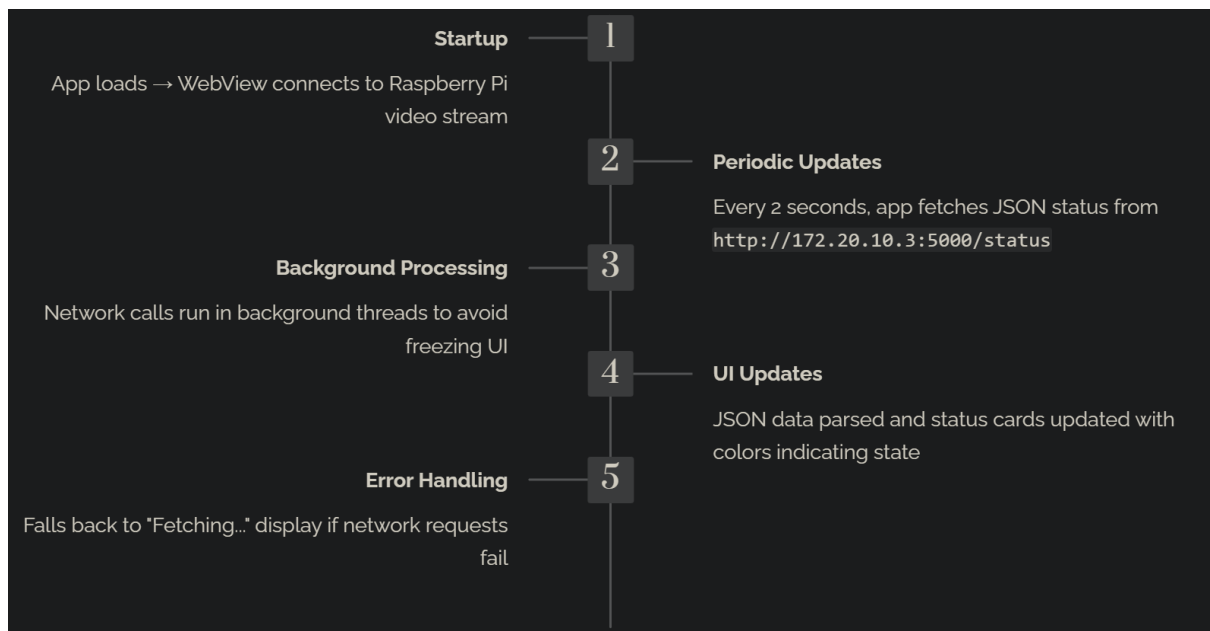


Fig. 8. User Interface (UI) of the Android Studio code displaying on pad



For Android Studio, this is used for both the communication between the pad and the layout configuration of the user interface. The main layout of the UI consists of two parts, where the left part is used to display the real-time video capturing of the camera sent from the Raspberry Pi, with the REFRESH and FULLSCREEN buttons for refreshing the connection and making the video stream full screen. Also, the connection IP Address is displayed in the bottom left for calibration. While for the right part, all the statuses of the system are displayed and updated at between certain intervals.

The main workflow for the system is that a mini server is first created by the PI, where the video streaming is sent to this webview, then the HTTP protocol is used for the information transmission between the PI and the Android Studio, with the app fetching the JSON status of the PI from the webview <http://172.20.10.3:5000/status> and displaying the real-time status.

activity_main.xml

```
<!-- MAIN LAYOUT: 60/40 split between video and status -->
<LinearLayout
    android:orientation="horizontal"
    android:layout_weight="6"> <!-- Left: 60% for video -->

    <WebView
        android:id="@+id/webView" <!-- Video streaming container -->
        android:layout_weight="1"/> <!-- Takes all vertical space -->

</LinearLayout>

<ScrollView
    android:layout_weight="4"> <!-- Right: 40% for status cards -->

    <androidx.cardview.widget.CardView <!-- Reusable card component -->
        <TextView android:id="@+id/doorStatusText"/> <!-- Dynamic status -->
```

Core UI Structures: Creates a responsive layout with video stream (60%) and real-time status cards (40%) using weight-based distribution as shown in Fig. 1.

AndroidManifest.xml

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />

<application android:usesCleartextTraffic="true"> <!-- Allow HTTP (not HTTPS) -->

    <activity android:name=".MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
```

Critical Permissions & Configuration: Grants internet access, allows HTTP connections to Raspberry Pi, and defines MainActivity as the app entry point.

network_security_config.xml

```
<domain-config cleartextTrafficPermitted="true">
    <domain includeSubdomains="true">172.20.10.3</domain>
</domain-config>
```

Security Exception: Allows unencrypted HTTP communication specifically with the Raspberry Pi

at IP 172.20.10.3 (My phone's hotspot IP address, may vary between different devices)

MainActivity.java

```
import android.webkit.WebView;           // Video streaming
import org.json.JSONObject;              // API response parsing
import java.net.HttpURLConnection;       // Network communication

private Handler statusHandler = new Handler();
private String piIpAddress = "172.20.10.3";
private Runnable statusUpdateTask;      // Periodic status updates
```

Critical Imports and Variables

```
private void setupWebView() {
    WebSettings webSettings = webView.getSettings();
    webSettings.setJavaScriptEnabled(true); // Enable web features
    webView.setWebViewClient(new WebViewClient()); // Handle page events
    loadVideoStream(); // Start video
}

private void loadVideoStream() {
    String streamUrl = "http://" + piIpAddress + ":5000";
    webView.loadUrl(streamUrl); // Load MJPEG stream
}
```

WebView Setup - Video Streaming

```
private Runnable statusUpdateTask = new Runnable() {
    @Override
    public void run() {
        fetchRealStatusData(); // Get data from Raspberry Pi
        statusHandler.postDelayed(this, 1000); // Repeat every 1 second
    }
};

private void fetchRealStatusData() {
    new Thread(new Runnable() { // Run in background thread
        public void run() {
            URL statusUrl = new URL("http://" + piIpAddress + ":5000/status");
            HttpURLConnection connection = (HttpURLConnection) statusUrl.openConnection();

            // Read JSON response from Raspberry Pi API
            BufferedReader in = new BufferedReader(new
InputStreamReader(connection.getInputStream()));
            StringBuilder response = new StringBuilder();
            while ((inputLine = in.readLine()) != null) {
                response.append(inputLine);
            }

            runOnUiThread(new Runnable() {
                public void run() {
                    parseAndUpdateStatus(jsonResponse); // Update UI with new data
                }
            });
        }
    }).start();
}
```

Real-time Status Updates

```
private void parseAndUpdateStatus(String jsonResponse) {
    JSONObject status = new JSONObject(jsonResponse); // Parse JSON

    // Update each status card with real data
    String doorStatus = status.getString("door_status");
    doorStatusText.setText(doorStatus.equals("unlocked") ? "Unlocked" : "Locked");
    doorStatusText.setTextColor(getResources().getColor(
        doorStatus.equals("unlocked") ? R.color.green : R.color.red));

    // Repeat for alarm, person detection, gestures, fan, LCD...
}
}
```

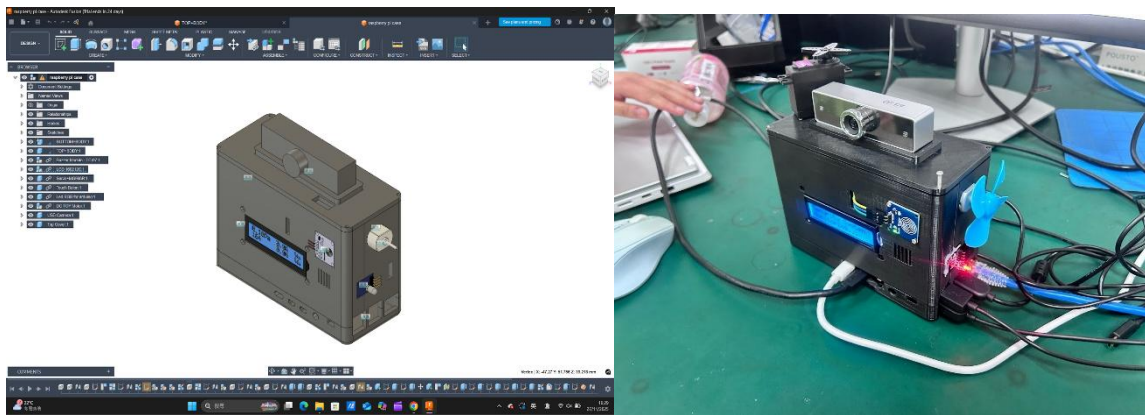
JSON Parsing and UI Updates

```
@Override
protected void onResume() {
    super.onResume();
    statusHandler.post(statusUpdateTask); // Start updates when app visible
}

@Override
protected void onPause() {
    super.onPause();
    statusHandler.removeCallbacks(statusUpdateTask); // Stop updates when app hidden
}
}
```

Activity Lifecycle Management

D. 3D Printed Case



After finishing the overall hardware connection of the system, a case is designed using Fusion 360, then the case is 3d printed and integrated with the whole system

Special Thanks to our classmate Xu Lenug Fai, DC326587, for borrowing his 3d printer for us to print the case.

III. CONCLUSIONS

In conclusion, this project has successfully developed a functional vision-based smart home control system, "GestureFace Smart Control," which integrates Raspberry Pi and Android Studio to achieve real-time monitoring, dual-identity, gesture authentication, and automated device control.

The primary challenges during development involved resolving library dependency conflicts on the

Raspberry Pi (we reinstalled the latest system on a new SD card to address this problem), establishing a stable, low-latency video stream between the hardware and the Android app, and significantly improving the low FPS issue. The latter was ingeniously addressed by implementing an alternating frame processing strategy for face and hand detection.

For future enhancements, we expect the system to be improved in several ways:

1. **Dynamic Face Recognition:** Implementing liveness detection to prevent the system from being spoofed by photographs.
2. **Cloud Integration & Notifications:** Adding cloud services to send real-time push notifications to the user's phone for specific events like security alarms.
3. **Voice Control:** Integrating a voice assistant (e.g., Google Assistant) to provide an alternative, hands-free control method alongside gestures.

IV. VIDEO FOOTAGE

For videos with English subtitles, please open the videos in the following link instead of the submitted video

[Donald Fong](#)

The four YouTube videos with the title "ECEN3024 project ..." in the following YouTube channel link, including the video introducing the basic information and the security checking of the system in both the outside view and the screen view of the pad UI.